



Docker Unleashed: Enhancing Local Environments with Ease

Subtitle

Kyle Jenkins

Kyle.Jenkins@improving.com

About me

I'm here to help people...

... and I love video games.



My Start with Docker

I hear it's cool, I should learn it.

Downloaded it for my project.

I ran this weird Docker command. Something is running.

Now what?

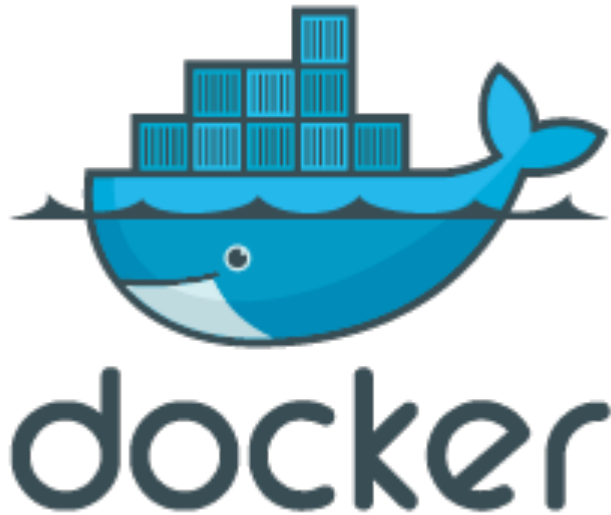
Client Switched from monolith to microservices utilizing docker

Locals ran it... though unstable. Anywhere from 20 minutes to 4 hours.

Over time, things clicked, made improvements, down to 5 minute start up.

Why talk about Docker?

From Local to Production



Beginner confusion

Is it the right tool?

Industry Standard

Why talk about Local Environments?

“Time is Money, Friend” – Every Goblin in World of Warcraft

Local initial Setups anywhere from 10 seconds to 5 Days

Visible changes anywhere from 2 seconds to 4 hours

Multiply that time between 3 and 40 people

Cost of inconsistencies

Cost of Frustrations

Cost of the Unknown



World of Warcraft®

Local Architecture: The User Stories

What do we need as developers?

Stand up environments: **minutes**

See visible changes: **seconds**

Confidence that if it works locally, it works across environments

Able to run/debug CI/CD pipeline jobs locally

Instantaneous Feedback

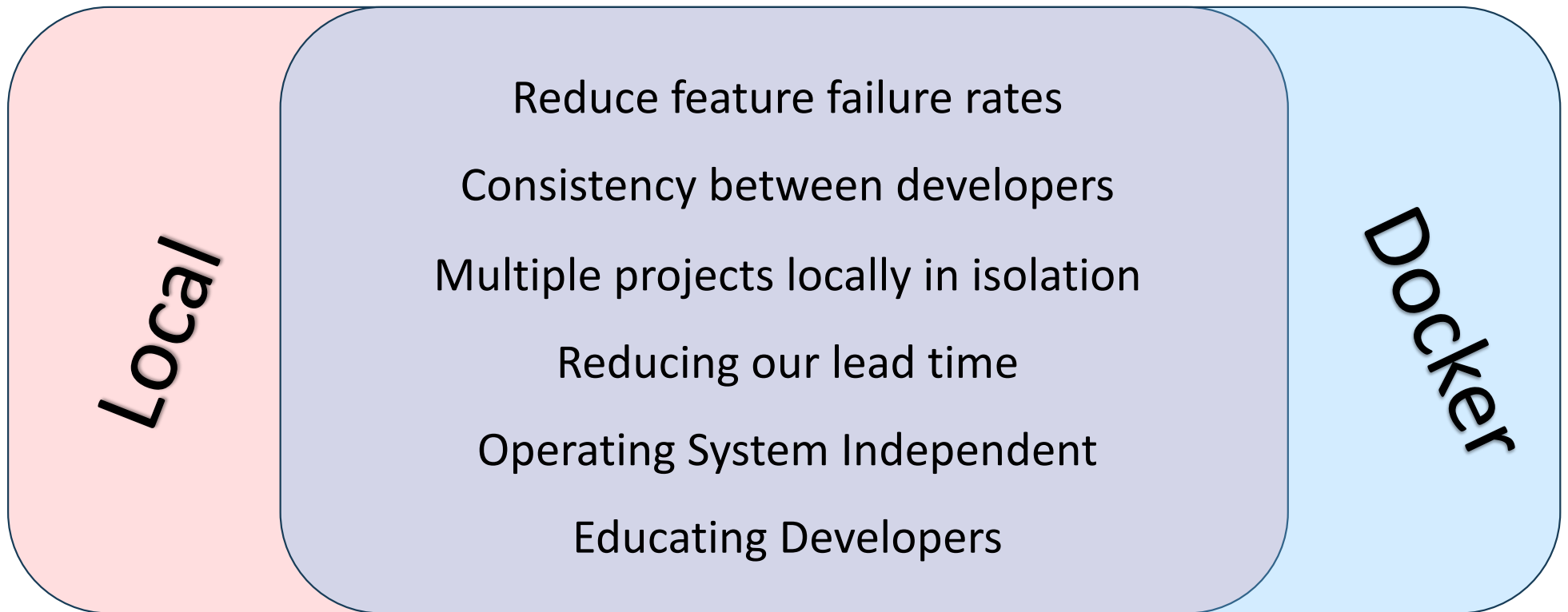
Flexibility per Developer



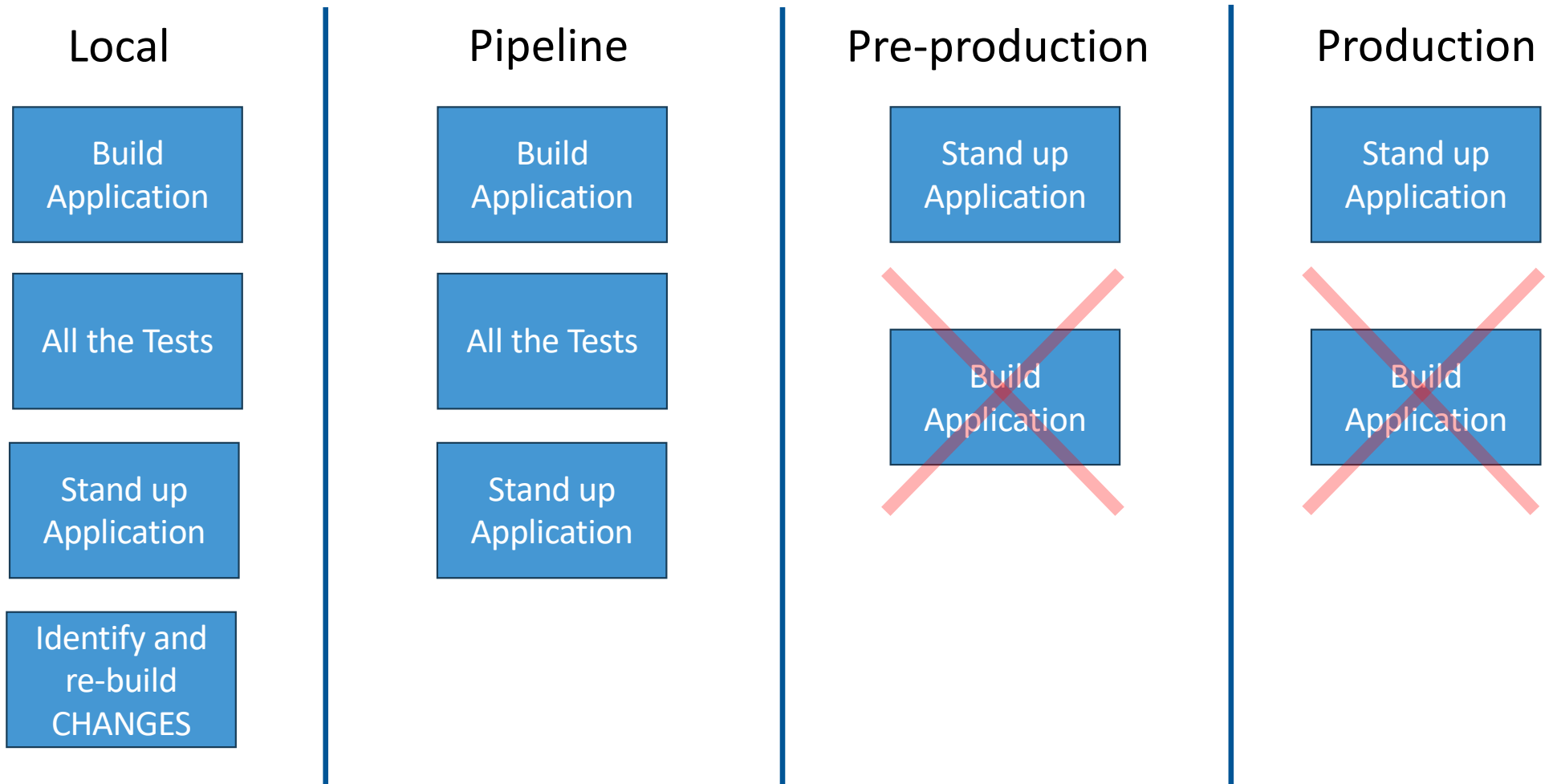
FINAL FANTASY XIV © 2010 - 2023 SQUARE ENIX CO., LTD. All Rights Reserved.

Putting them Together

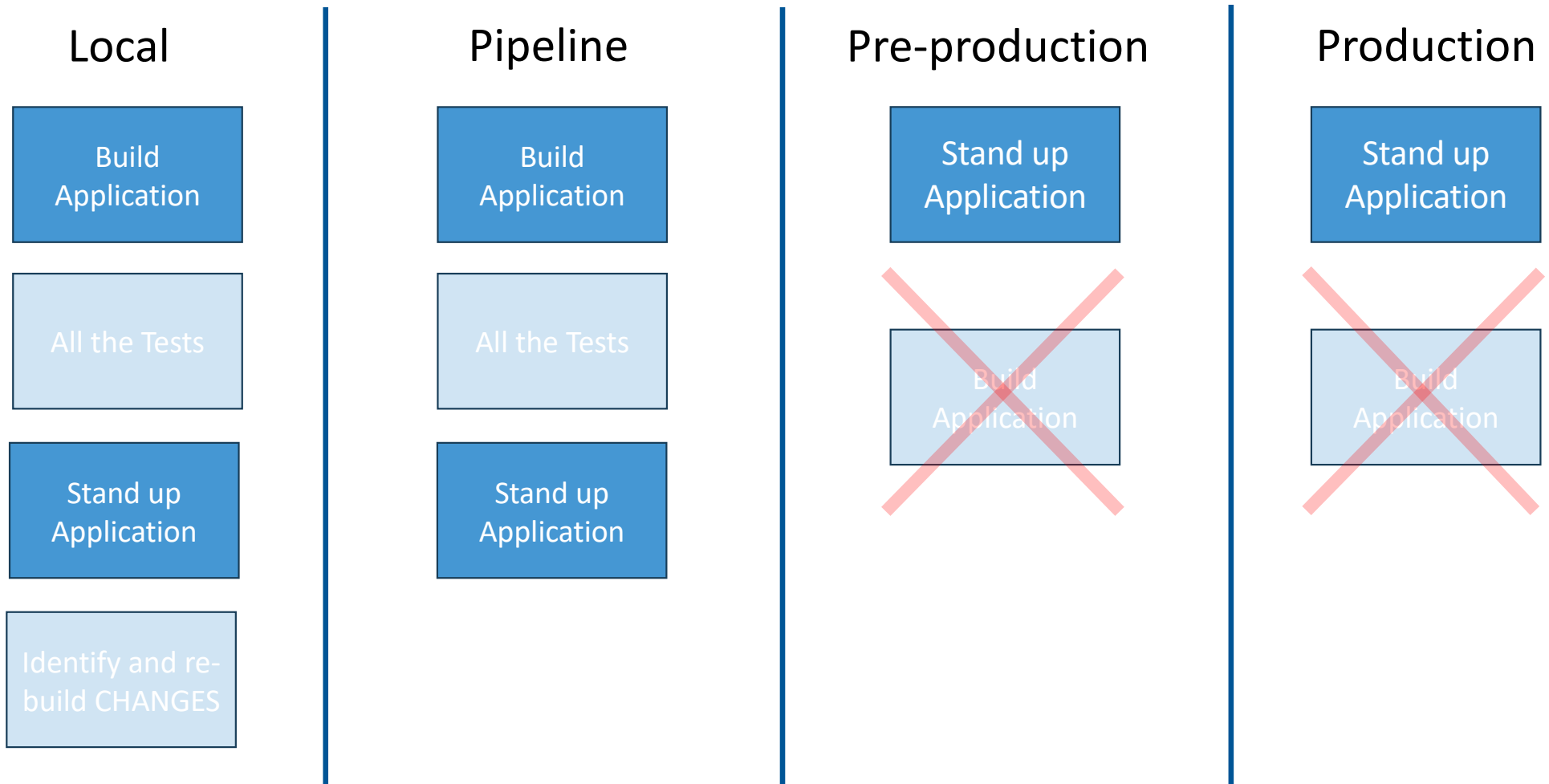
What can we get?



High Level Environment View: Local to Production

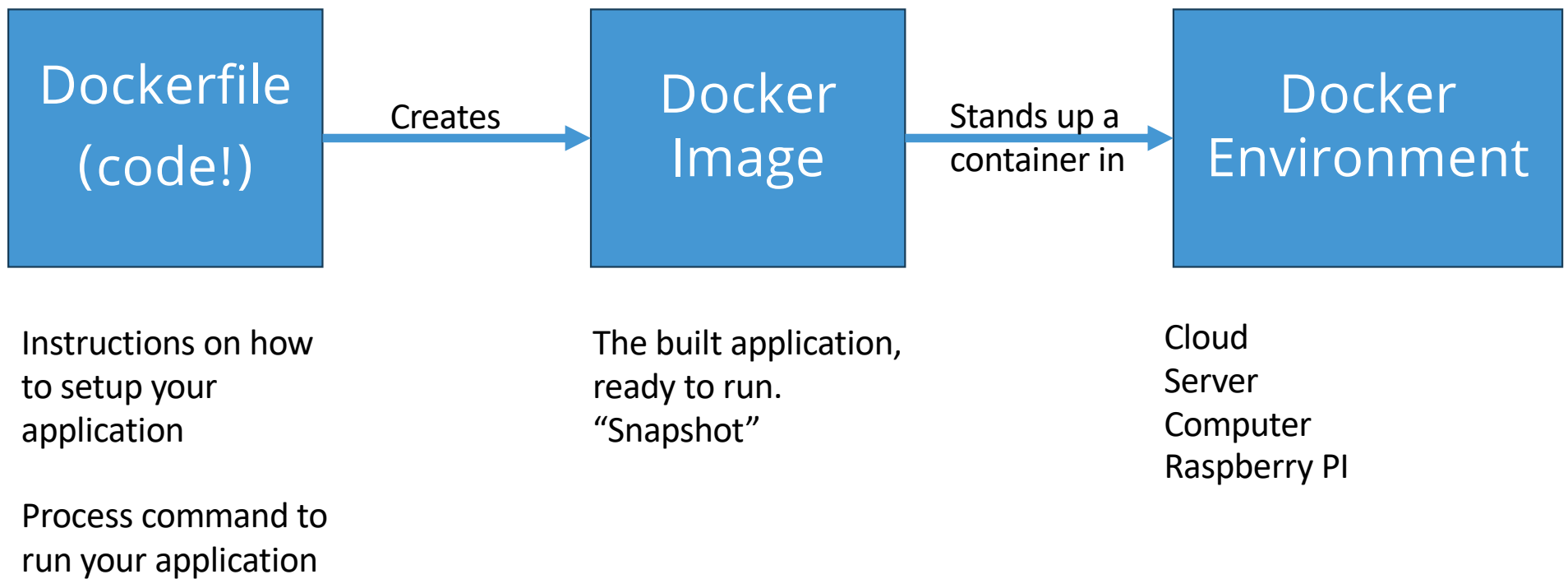


Where does Docker come into this?

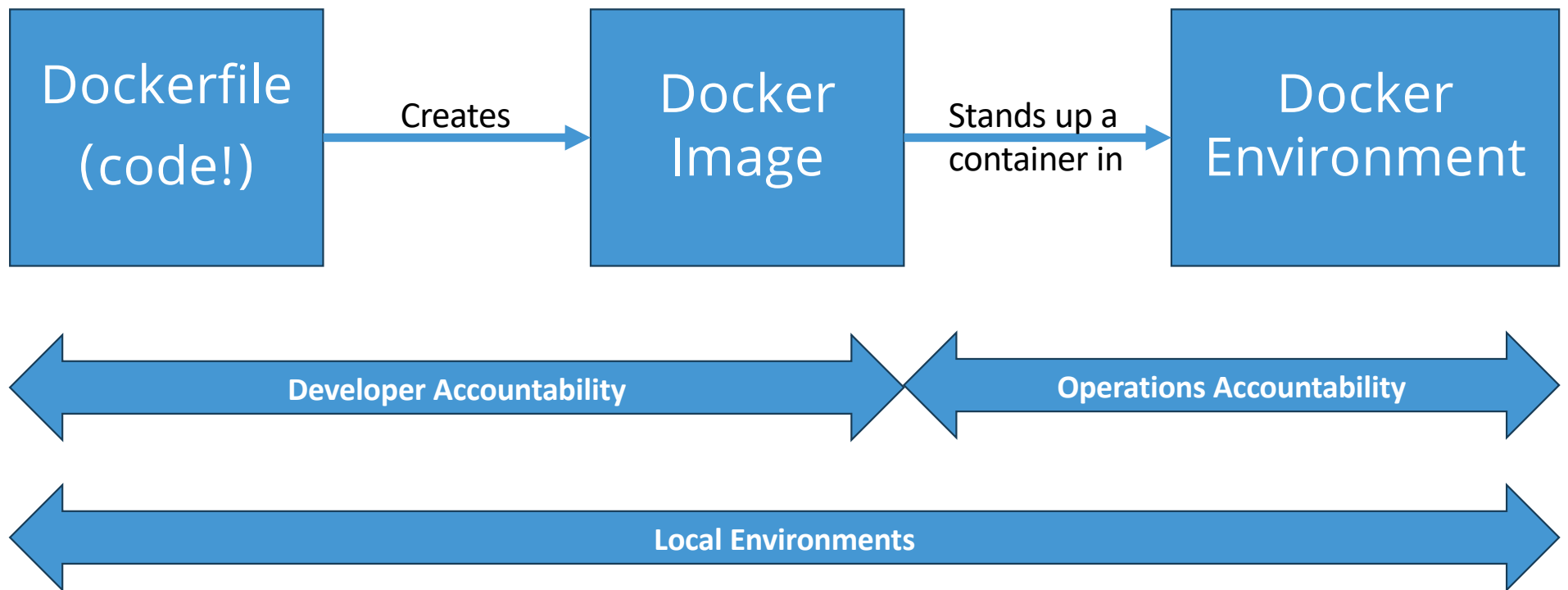


My one slide intro to Docker!

From the perspective of a coder



Accountability



When should I use Docker locally?

(Note: these are my own biases, there are always caveats, consider your own needs)

Application in Cloud? 	Creating a basic library / package? 
Static Site? 	Testing your library / package? 

Your Local Computer Everything gets complex

Compu tainer
Even
ing.?



Your Local Computer: During Development

Computer

~~Application~~
Build Process
Watch
Code Linter
Tests
IDE

Container

Application
Build Process
Watch
~~Code Linter~~*
~~Tests~~*
~~IDE~~

Your Local Computer: What if I want “production-like”?

Computer

~~Application~~
Build Process
~~Watch~~
Code Linter
Tests
IDE

Container

Application
~~Build Process~~
~~Watch~~
~~Code Linter~~
~~Tests~~
~~IDE~~

Dependencies vs. Development Dependencies

NodeJS: Package.json Example

Development Only

```
{  
  "devDependencies": {  
    "eslint": "^8.48.0",  
    "jest": "^29.6.4",  
    "nodemon": "^3.0.1"  
  }  
}
```

Production. Verify these heavily.

```
{  
  "dependencies": {  
    "express": "^4.18.2",  
    "react": "^18.2.0",  
    "react-dom": "^18.2.0"  
  }  
}
```


Example #1

Single Dockerfile

Example Dockerfile: Simple (dev/prod)

```
FROM node:18.17.1          # Version of Node. Be specific. Use your Image Repository

ARG NODE_ENV                # Provide the Node Environment (Production | Development)

WORKDIR /usr/src/app        # What is my working directory?

COPY package.json ./        # Which packages to install
COPY package-lock.json ./   # Defines exact versions to install
RUN npm ci                  # Installs exact versions. Replaces `npm install`

COPY . .                    # Copy the runtime/built code. Need .dockerignore if using `.`

USER node                   # Don't run as root!!!!

EXPOSE 6080                 # Documentation only, which ports are we expected to have exposed
CMD [ "node", "index.js" ] # the command being run
```

.dockerignore

Exclude List Style

```
.idea  
.vscode  
node_modules  
dist
```

Include List Style

```
# Exclude All  
*  
  
# Intentional Includes  
!package.json  
!package-lock.json  
!src  
!index.js
```

Example Docker-Compose

services:

```
my-app:                                     # Reference name of my service locally
  image: my-app:latest                     # Name your Docker Image, probably repository name
  build:
    context: .                             # What folder should the build happen from?
    dockerfile: Dockerfile                 # The Dockerfile.
    args:
      - NODE_ENV=development               # The Node Environment we specified
  ports:
    - "52416:6080"
  volumes:                                # Sync any runtime files
    - "./src:/usr/src/app/src"
    - "./dist:/usr/src/app/dist"
  environment:
    - NODE_ENV=development
  command: ["npm", "run", "dev"]           # Runtime command for development
```

Running it all

```
$ docker compose up -d --build
```

-d: Run in the background

--build: Build the Docker file first

```
$ docker compose ps
```

Check status

```
$ docker compose down
```

Stop and remove

```
$ docker compose logs -f
```

Watch logs

```
$ docker compose exec my-app /bin/bash
```

SSH in

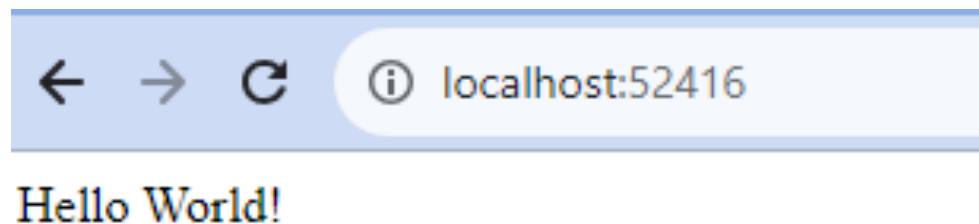
Example (Demo)

```
my-express-app>
```

```
// src/index.js
const express = require('express')
const app = express()

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(6080, () => {
  console.log('Example app listening on port 6080')
})
```



Example “Production-like” Docker-Compose

```
# --- docker-compose.production.yml ---
services:
  my-app-prod:
    image: my-app:latest
    build:
      args:
        - NODE_ENV=production
    ports:
      - "6112:6080"
    environment:
      - NODE_ENV=production
    # Reference name of my service locally
    # Name your Docker Image, probably repository name
    # The Node Environment we specified

# --- Commands ---
$ NODE_ENV=production npm run build
$ docker-compose up -f docker-compose.production.yml -d --build
```


Pipelines

Step by Step, Keeping it Simple

Running Tests

image: node:18.17.1

cmd: npm ci

Install Dependencies

cmd: npm run build

cmd: npm run test

Always run tests!!!

Pipelines

Step by Step, Keeping it Simple

Building the Image

```
$ docker build  
  --build-arg NODE_ENV=production  
  -t <my_registry>/my-app:latest
```

Push the image up

```
$ docker push <my_registry>/my-app:latest
```

Environment Variable Management

Put the Developers Accountable!

Production: Default

Pre-prod: Default

Pipelines: Override

Local: Override

(others): Override

```
// Myfile.js:
```

```
const {  
  NODE_ENV = 'production',  
  myApiUrl = 'http://myApi:80',  
} = process.env;
```

Misconception: NOT for changing variables during runtime

Example #2

Development Only: Build, Watch and Serve
Create React App

Example Dockerfile: Create React App

```
FROM node:18.17.1
```

```
ARG NODE_ENV
```

```
WORKDIR /usr/src/app
```

```
COPY package.json ./
```

```
COPY package-lock.json ./
```

```
RUN npm ci
```

```
# Needed for create react app scripts
```

```
RUN mkdir -p ./node_modules/.cache && chown node:node ./node_modules/.cache
```

```
COPY . .
```

```
USER node
```

```
EXPOSE 3000
```

```
CMD [ "npm", "start" ]
```

Example Docker-compose: Create React App

```
version: "3.7"
```

```
services:
```

```
  my-app:
```

```
    image: my-app:latest
```

```
    build:
```

```
      args:
```

```
        - NODE_ENV=development
```

```
    ports:
```

```
      - "1325:3000"
```

```
    volumes:
```

```
      - "./src:/usr/src/app/src"
```

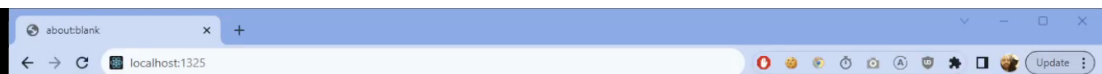
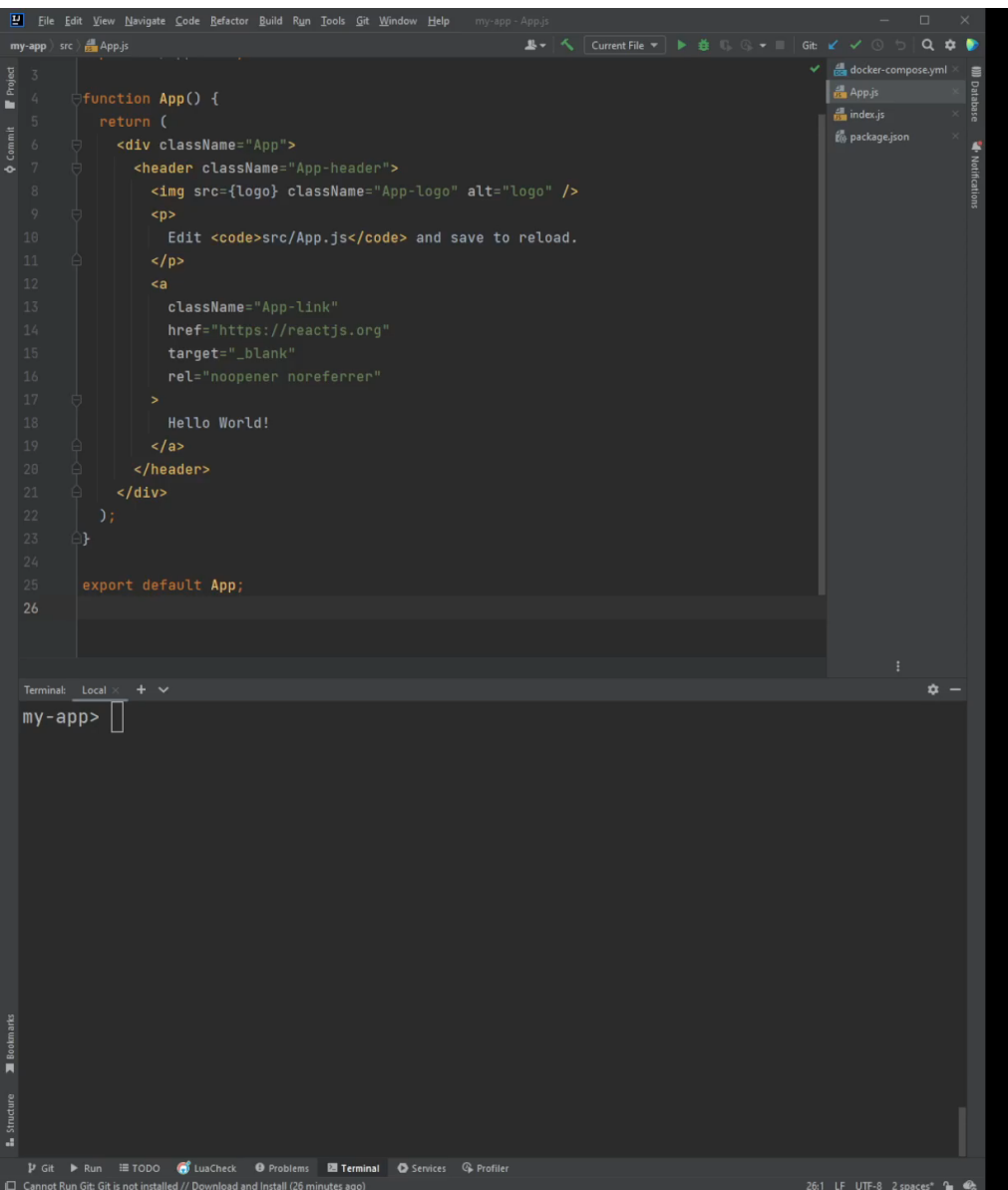
```
    environment:
```

```
      - NODE_ENV=development
```

```
      - CHOKIDAR_USEPOLLING=true    # < react scripts 5
```

```
      - WATCHPACK_POLLING=true     # >= react scripts 5
```

```
      - WDS_SOCKET_PORT=1325       # The mapped port on your host machine
```



Example 3: Static Sites / CDN Example

Serve Only

NGINX

Static Sites / CDN

Computer

~~Application~~
Build Process
Watch
Code Linter
Tests
IDE

Container

Application
~~Build Process~~
~~Watch~~
~~Code Linter~~
~~Tests~~
~~IDE~~

Example Docker-Compose: NGINX

```
version: "3.7"
```

```
services:
```

```
  static-app:                                # Reference name of my service locally
    image: nginx:latest                      # Name of Image to use
    ports:
      - "19200:80"
    volumes:
      - "./dist:/usr/share/nginx/html"      # Sync your built static files
```

```
# Commands
```

```
$ docker-compose up -d
```

Example Combining it all

```
version: "3.7"
```

```
services:
```

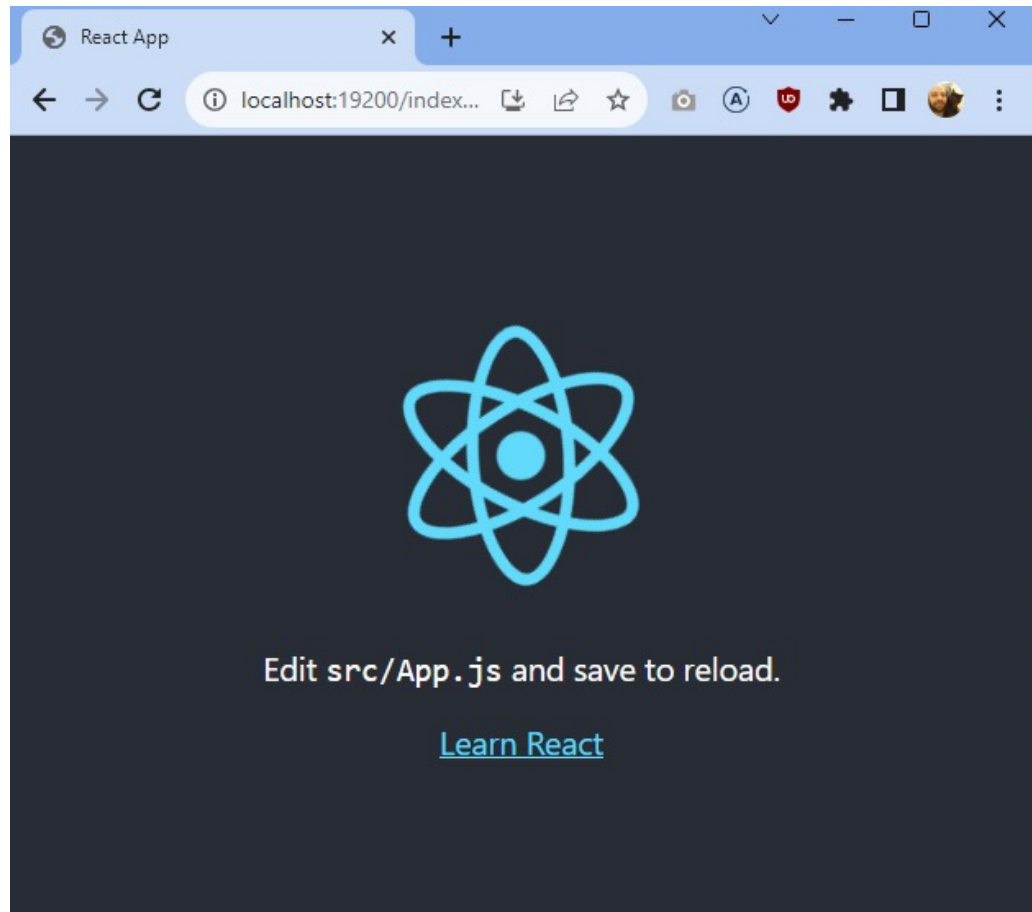
```
static-app:                                # Reference name of my service locally
  image: nginx:latest                      # Name of Image to use
  ports:
    - "19200:80"
  volumes:
    - "./dist:/usr/share/nginx/html"      # Sync your built static files
```

```
# Commands
```

```
$ npm run build
```

```
$ docker-compose -f docker-compose.prod.yml up -d
```

Example Docker-Compose: NGINX



Example 4: Multistage Dockerfiles

Including the Build Process

Before Example: EC2 Static Angular Instance

Inside of an EC2 Ubuntu Server

Install git

Download the repository

Install Node

Install packages (npm ci)

Build Static Files

Install NGINX

Copy Static files to static http directory

Serve port 80

Problems

- Dev dependencies installed
- Unnecessary tools installed
- Larger server than needed
- Running as root

Multistage Dockerfiles: NGINX

```
FROM node:18.17.1 as builder                # Container Doing the Build
...
RUN npm build                               # Builds the application

FROM nginxinc/nginx-unprivileged:stable-alpine # Provided by NGINX

COPY --from=builder /build-output /usr/share/nginx/html # copy from builder

EXPOSE 8080

CMD ["nginx", "-g", "daemon off;"]
```

Taking it up a notch

A bit more advanced stuff

Libraries? Similar to Static

Computer

~~Application~~
Build Process
Watch
Code Linter
Tests
IDE

Container

Test Application
~~Build Process~~
~~Watch~~
~~Code Linter~~
~~Tests~~
~~IDE~~

More than one process? More than one container!

```
version: "3.7"
```

```
services:
```

```
  node-app:
```

```
    # Reference name of my service locally
```

```
    image: node:18.17.1
```

```
    ports:
```

```
      - "19200:80"
```

```
    env:
```

```
      - CACHE_HOST: "cache:19201"
```

```
    # Remember default to standard port in prod environment
```

```
  cache:
```

```
    image: redis:7.2.0
```

```
    ports:
```

```
      - "19201:80"
```

Additional Dockerfiles

Dockerfile

Development.dockerfile

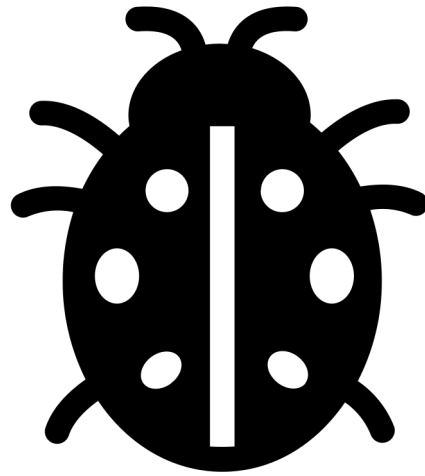
WebService.dockerfile

Dockerfile.jvm

Dockerfile.native

Debugging

Check your IDE Documentation



Alternatives

Because businesses change

- Podman
- Buildah
- BuildKit
- Kaniko
- Skopeo
- Dive
- Runc and crun
- LXD
- containerd

```
# .bashrc  
alias docker=podman
```

Summary

Docker is a great tool to have in the toolbox. Use wisely.



Utilizing Docker from Local to Production can assist with

- Consistency
- Reduce feature failure rate
- Improve our lead time of features



Encourage Developer Education



Focus pain points for the team



Time is Money! Do not ignore your local environments!!!!





If you're application works, you're not doing wrong. There is, however, always room for improvement.

Kyle Jenkins

